

INFININOTE - KIẾN TRÚC CLOUD MULTI-DEVICE 0 ĐỒNG

Mục tiêu tài liệu: Thiết lập một hệ thống ứng dụng Canvas vô hạn (Whiteboard) cá nhân hoạt động liên tục 24/7 trên môi trường Internet mà không tốn bất kỳ chi phí duy trì nào, tối ưu hóa các hạn mức miễn phí (Free Tier) từ các nhà cung cấp dịch vụ Cloud hiện đại.

1. Mô Hình Tổng Quan Hệ Thống (System Architecture)

Hệ thống hoạt động theo mô hình Client-Server phi tập trung sử dụng dịch vụ đám mây đám mây bên thứ ba. Dữ liệu được đồng bộ hóa thời gian thực (Real-time Synchronization) qua giao thức WebSockets.

Thành phần	Dịch vụ lựa chọn (Free Tier)	Vai trò & Giới hạn miễn phí
Giao diện (Frontend)	Vercel / Netlify	Hosting ứng dụng Web/PWA (React + tldraw). Miễn phí băng thông 100GB/tháng, tự động build từ GitHub.
Máy chủ (Backend)	Render / Koyeb	Chạy runtime Python (FastAPI). Hỗ trợ WebSockets kết nối liên tục. Tự động ngủ khi không có traffic (tiết kiệm tài nguyên).
Cơ sở dữ liệu	MongoDB Atlas	Lưu cấu trúc các Node đồ họa dạng BSON/JSON. Gói M0 miễn phí 512MB dung lượng lưu trữ (đủ cho hàng triệu node text).
Kho lưu trữ ảnh	Supabase Storage / Cloudinary	Lưu trữ các tệp ảnh/tài liệu được dán (paste) từ Clipboard. Supabase miễn phí 1GB lưu trữ, Cloudinary miễn phí 25GB biến đổi media.

2. Thiết Kế Cơ Sở Dữ Liệu (Data Model)

Do cấu trúc của một Canvas vô hạn gồm nhiều thành phần hình học (Text, Rectangle, Line, Image) có tọa độ x, y linh hoạt, cấu trúc tài liệu NoSQL của MongoDB là lựa chọn tối ưu nhất. Mỗi Canvas là một Document chứa một mảng các đối tượng (Shapes).

```
{
  "_id": "board_user_01",
  "updated_at": "2026-06-24T04:00:00Z",
  "shapes": [
    {
      "id": "shape_01",
```

```

        "type": "text",
        "x": 120.5,
        "y": -45.2,
        "props": { "text": "Số đồ Quy trình Công việc", "color": "purple" }
    },
    {
        "id": "shape_02",
        "type": "image",
        "x": 300.0,
        "y": 150.0,
        "props": { "url": "https://res.cloudinary.com/.../img.png", "w": 400, "h": 250 }
    }
]
}

```

3. Khung Mã Nguồn Máy Chủ (FastAPI Backend Starter)

Dưới đây là đoạn code Python chuẩn hóa sử dụng FastAPI phối hợp với thư viện motor (driver MongoDB bất đồng bộ) hỗ trợ đồng bộ dữ liệu qua WebSocket:

```

from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from motor.motor_asyncio import AsyncIOMotorClient
import json

app = FastAPI(title="InfiniNote Core Server")

# Kết nối MongoDB Atlas (Thay chuỗi kết nối bằng URI miễn phí của bạn)
MONGO_URI = "mongodb+srv://<username>:<password>@cluster.mongodb.net/test"
client = AsyncIOMotorClient(MONGO_URI)
db = client["infininote_db"]
board_collection = db["canvas_boards"]

# Quản lý danh sách các thiết bị đang kết nối trực tuyến
class ConnectionManager:
    def __init__(self):
        self.active_connections: list[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)

    async def broadcast(self, message: str, exclude_ws: WebSocket = None):
        for connection in self.active_connections:
            if connection != exclude_ws:
                await connection.send_text(message)

manager = ConnectionManager()

```

```

@app.websocket("/ws/{board_id}")
async def websocket_endpoint(websocket: WebSocket, board_id: str):
    await manager.connect(websocket)

    # Tải trạng thái canvas cũ từ database gửi cho thiết bị mới kết nối
    board_data = await board_collection.find_one({"_id": board_id})
    if board_data:
        await websocket.send_text(json.dumps({"type": "INIT_LOAD", "shapes":
board_data.get("shapes", [])}))

    try:
        while True:
            data = await websocket.receive_text()
            payload = json.loads(data)

            # Cập nhật thay đổi vào MongoDB
            await board_collection.update_one(
                {"_id": board_id},
                {"$set": {"shapes": payload.get("shapes", [])}},
                upsert=True
            )

            # Phát tán sự thay đổi tọa độ đến tất cả các thiết bị khác ngay lập tức
            await manager.broadcast(data, exclude_ws=websocket)
    except WebSocketDisconnect:
        manager.disconnect(websocket)

```

4. Quy Trình Các Bước Triển Khai Thực Tế

1. **Khởi tạo Cơ sở dữ liệu:** Đăng ký tài khoản MongoDB Atlas miễn phí, tạo một Cluster M0, thiết lập Network Access cho phép truy cập (IP 0.0.0.0/0) và lấy chuỗi Connection URI.
2. **Đẩy mã nguồn lên GitHub:** Chia mã nguồn của bạn thành 2 thư mục độc lập: /frontend (chứa mã nguồn React + Tldraw) và /backend (chứa code FastAPI). Đẩy toàn bộ lên một kho lưu trữ GitHub cá nhân (Private hoặc Public).
3. **Triển khai Server Backend:** Đăng ký tài khoản Render. Tạo một dịch vụ mới dạng **Web Service**, kết nối với kho lưu trữ GitHub của bạn, chọn môi trường Python, thiết lập lệnh khởi chạy (Start Command) là `uvicorn main:app --host 0.0.0.0 --port $PORT` và thêm biến môi trường `MONGO_URI`.
4. **Triển khai Giao diện Frontend:** Đăng ký tài khoản Vercel. Tạo project mới liên kết với GitHub, trở vào thư mục /frontend và nhấn Deploy. Cấu hình biến môi trường kết nối WebSocket trở về địa chỉ URL mà Render vừa cấp.
5. **Cài đặt PWA lên Thiết bị Di động:** Truy cập vào đường link Vercel bằng trình duyệt Safari (iOS) hoặc Chrome (Android), chọn mục *Chia sẻ / Tùy chọn* -> bấm **Thêm vào màn hình chính (Add to Home Screen)**. Ứng dụng sẽ xuất hiện ngoài màn hình điện thoại với giao diện toàn màn hình cực kỳ mượt mà.